

Finding CB Numbers

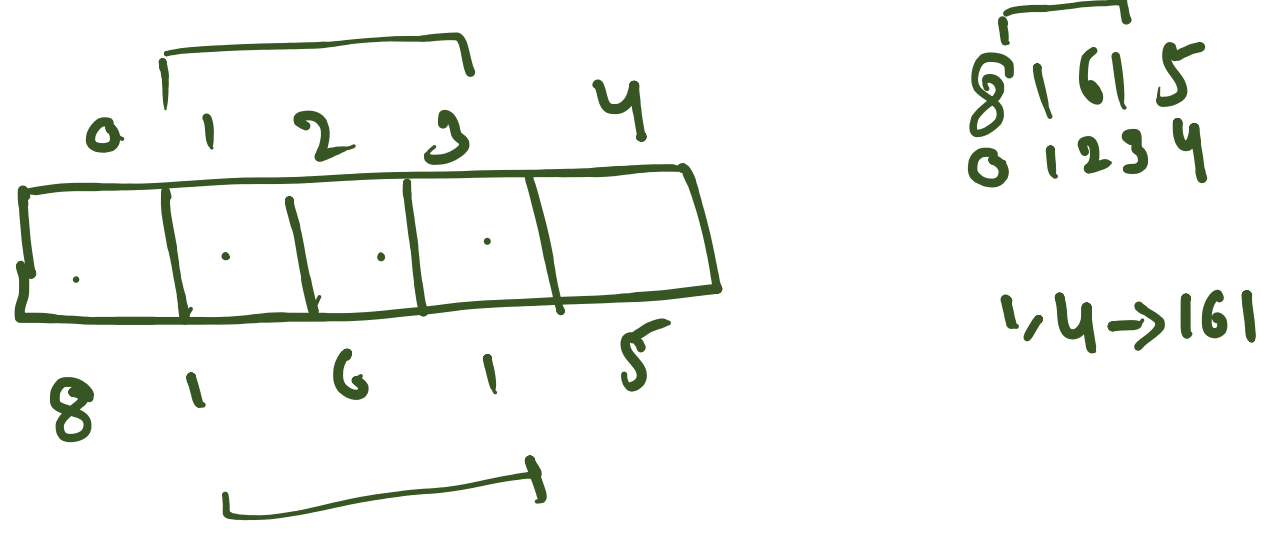
Kartik Bhaiya and Kanak Bhaiya are discussing a special type of number, which they call a **Coding Blocks Number (CB Number)**. A number qualifies as a CB Number based on the following criteria:

- 0 and 1 are not considered CB numbers.
- The following prime numbers are always CB numbers:
2, 3, 5, 7, 11, 13, 17, 19, 23, 29
- Any other number is a CB number if it is not divisible by any of the numbers listed in point (2).

Handwritten notes and calculations:

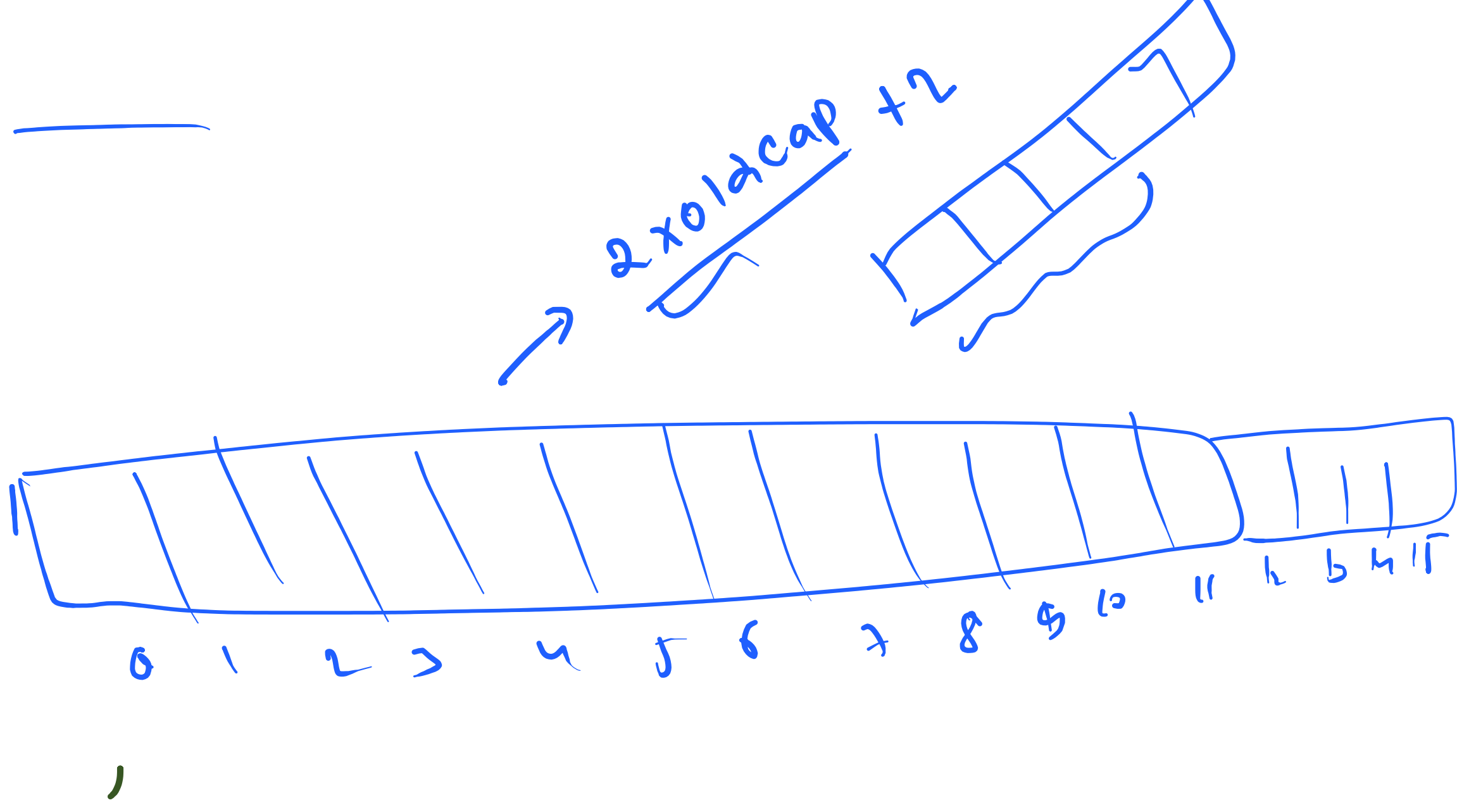
- $37 \times 41 = 1517$ (boxed)
- $99 \times 99 = 9801$ (circled)
- 11991 (circled)
- 499 (circled)
- 991 (circled)
- 5 (circled)
- 217 (circled)
- 127 (circled)
- $127 \times 1 = 127$
- $127 \times 2 = 254$
- $127 \times 3 = 381$
- $127 \times 4 = 508$
- $127 \times 5 = 635$
- $127 \times 6 = 762$
- $127 \times 7 = 889$
- $127 \times 8 = 1016$
- $127 \times 9 = 1143$
- $127 \times 10 = 1270$
- $127 \times 11 = 1397$
- $127 \times 12 = 1524$
- $127 \times 13 = 1651$
- $127 \times 14 = 1778$
- $127 \times 15 = 1905$
- $127 \times 16 = 2032$
- $127 \times 17 = 2159$
- $127 \times 18 = 2286$
- $127 \times 19 = 2413$
- $127 \times 20 = 2540$
- $127 \times 21 = 2667$
- $127 \times 22 = 2794$
- $127 \times 23 = 2921$
- $127 \times 24 = 3048$
- $127 \times 25 = 3175$
- $127 \times 26 = 3302$
- $127 \times 27 = 3429$
- $127 \times 28 = 3556$
- $127 \times 29 = 3683$
- $127 \times 30 = 3810$
- $127 \times 31 = 3937$
- $127 \times 32 = 4064$
- $127 \times 33 = 4191$
- $127 \times 34 = 4318$
- $127 \times 35 = 4445$
- $127 \times 36 = 4572$
- $127 \times 37 = 4699$
- $127 \times 38 = 4826$
- $127 \times 39 = 4953$
- $127 \times 40 = 5080$
- $127 \times 41 = 5207$
- $127 \times 42 = 5334$
- $127 \times 43 = 5461$
- $127 \times 44 = 5588$
- $127 \times 45 = 5715$
- $127 \times 46 = 5842$
- $127 \times 47 = 5969$
- $127 \times 48 = 6096$
- $127 \times 49 = 6223$
- $127 \times 50 = 6350$
- $127 \times 51 = 6477$
- $127 \times 52 = 6604$
- $127 \times 53 = 6731$
- $127 \times 54 = 6858$
- $127 \times 55 = 6985$
- $127 \times 56 = 7112$
- $127 \times 57 = 7239$
- $127 \times 58 = 7366$
- $127 \times 59 = 7493$
- $127 \times 60 = 7620$
- $127 \times 61 = 7747$
- $127 \times 62 = 7874$
- $127 \times 63 = 8001$
- $127 \times 64 = 8128$
- $127 \times 65 = 8255$
- $127 \times 66 = 8382$
- $127 \times 67 = 8509$
- $127 \times 68 = 8636$
- $127 \times 69 = 8763$
- $127 \times 70 = 8890$
- $127 \times 71 = 9017$
- $127 \times 72 = 9144$
- $127 \times 73 = 9271$
- $127 \times 74 = 9398$
- $127 \times 75 = 9525$
- $127 \times 76 = 9652$
- $127 \times 77 = 9779$
- $127 \times 78 = 9906$
- $127 \times 79 = 10033$
- $127 \times 80 = 10160$
- $127 \times 81 = 10287$
- $127 \times 82 = 10414$
- $127 \times 83 = 10541$
- $127 \times 84 = 10668$
- $127 \times 85 = 10795$
- $127 \times 86 = 10922$
- $127 \times 87 = 11049$
- $127 \times 88 = 11176$
- $127 \times 89 = 11303$
- $127 \times 90 = 11430$
- $127 \times 91 = 11557$
- $127 \times 92 = 11684$
- $127 \times 93 = 11811$
- $127 \times 94 = 11938$
- $127 \times 95 = 12065$
- $127 \times 96 = 12192$
- $127 \times 97 = 12319$
- $127 \times 98 = 12446$
- $127 \times 99 = 12573$
- $127 \times 100 = 12700$

```
public static int Count_Cb_Number(String str) {
    int c = 0;
    for (int len = 1; len <= str.length(); len++) {
        for (int j = len; j <= str.length(); j++) {
            int i = j - len;
            String s = str.substring(i, j);
            if (Is_CB_Numbers(Long.parseLong(s))) {
                c++;
            }
        }
    }
    return c;
}
```



```
public static int Count_Cb_Number(String str) {
    int c = 0;
    boolean[] visited = new boolean[str.length()];
    for (int len = 1; len <= str.length(); len++) {
        for (int j = len; j <= str.length(); j++) {
            int i = j - len;
            String s = str.substring(i, j);
            if (Is_CB_Numbers(Long.parseLong(s)) && !visited[i]) {
                c++;
                visited[i] = true;
            }
        }
    }
    return c;
}

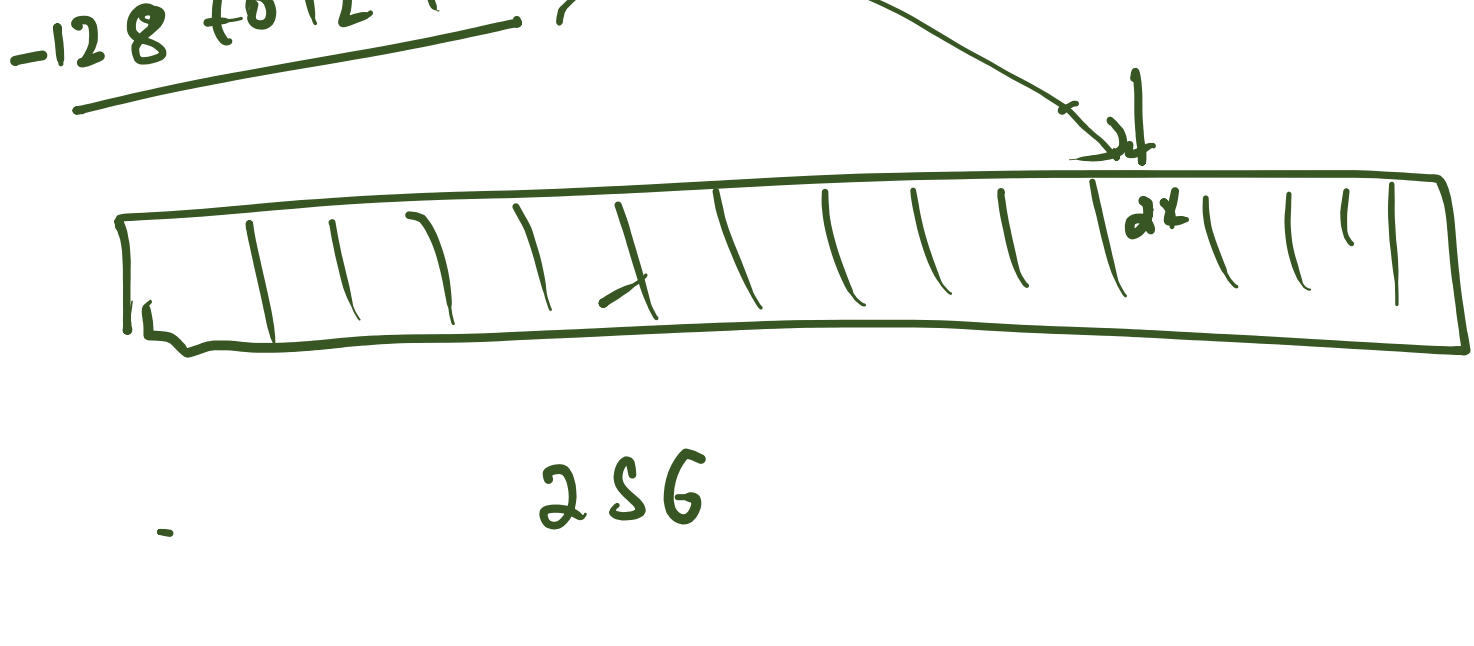
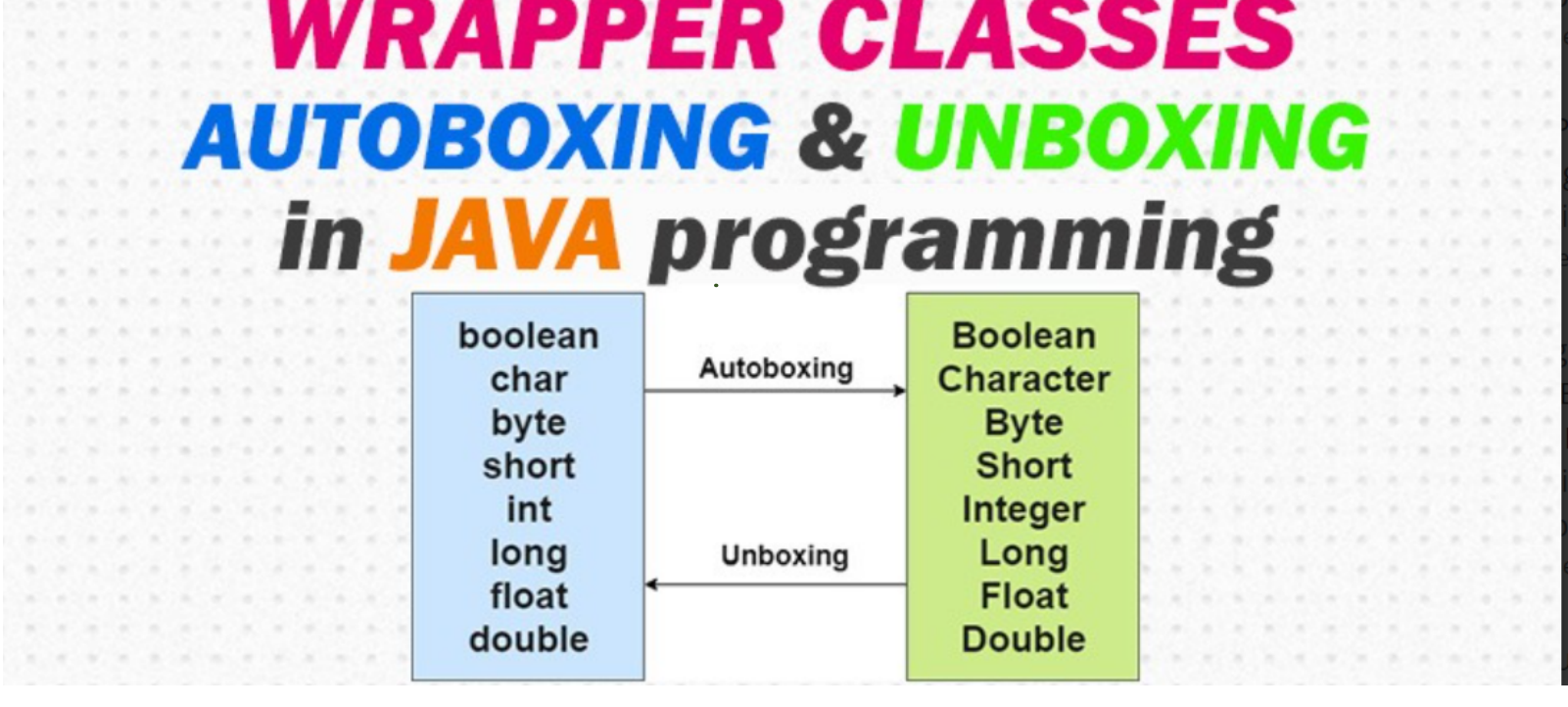
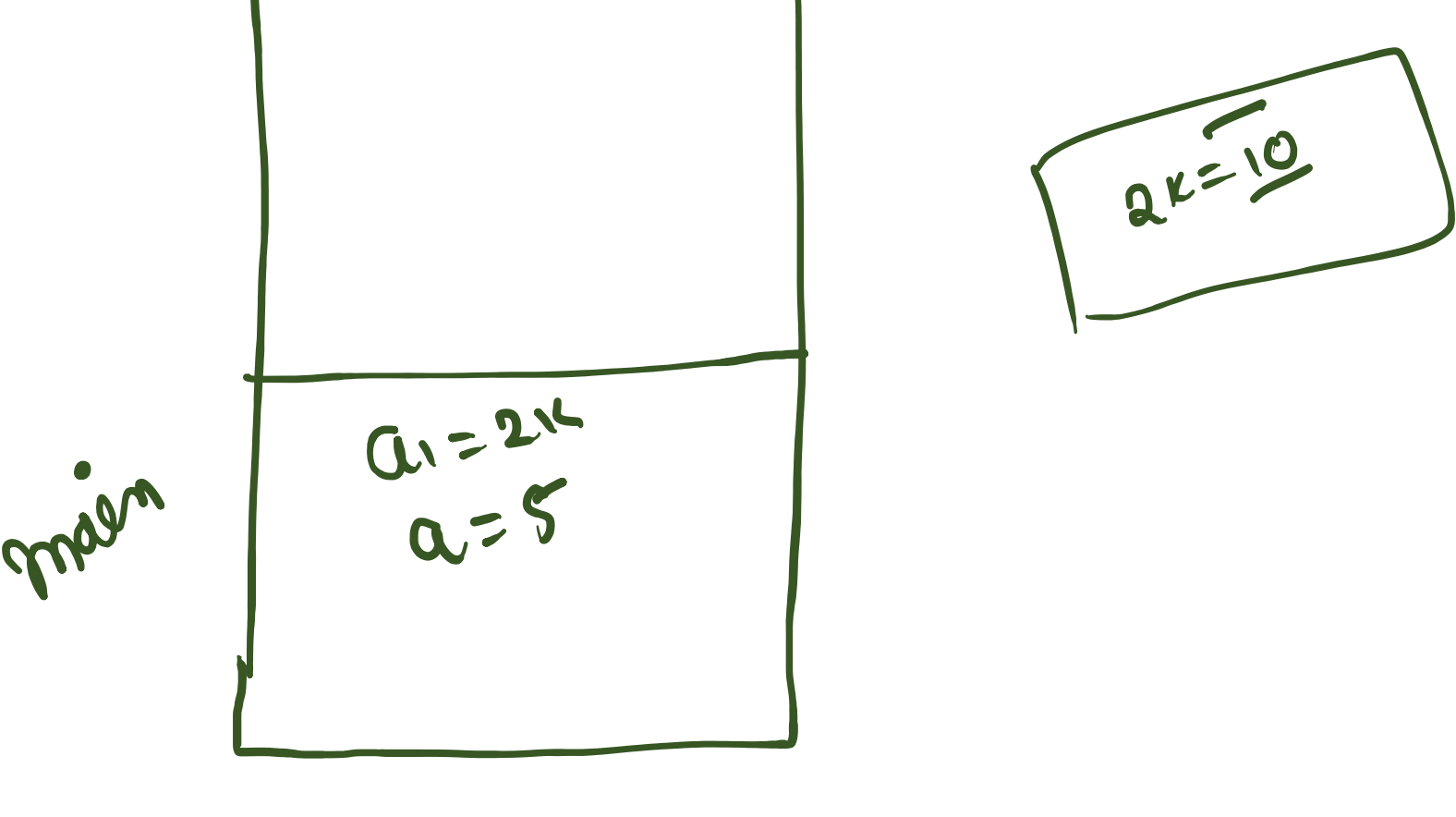
public static boolean isvisited(boolean[] visited, int i, int j) {
    // TODO Auto-generated method stub
    for (int k = i; k <= j; k++) {
        if (visited[k]) {
            return true;
        }
    }
    return false;
}
```



Wrapper class
↳ primitive
→ 8
Integer

Primitive data type	Wrapper Class
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean

```
public static void main(String[] args) {
    // TODO Auto-generated method stub
    Integer a1=10;
    int a=5;
}
```



$(p - a) = 15$
↳ array code (memory)
 $ch \rightarrow 'a' \rightarrow 97 - 97 = 0$
 $k \rightarrow 107 - 97 = 10$

